Image-to-Sentence Retrieval via CNN Embedding Regression: A Multi-Dataset Comparative Study

Fergus Watson **Gia Wang** **Seoyeon Kim** **Sujith Goli** **Zhenmao Song**
jn25200@bristol.ac.uk vu25289@bristol.ac.uk hv25324@bristol.ac.uk iz25057@bristol.ac.uk fc25028@bristol.ac.uk
https://github.com/Rylie-KIM/uob-ds-intro-to-ai-final-cw-2026

Abstract

This project addresses image-to-sentence retrieval. Given a query image, a CNN encoder maps it into a sentence embedding space, and the corpus sentence with the highest cosine similarity is returned. We implemented three structurally different datasets: synthetic shape-colour descriptions (Type-A), coloured MNIST digit sequences (Type-B), and tic-tac-toe board states (Type-C). We investigated the optimal combination of visual encoder, text representation, and training objective, which varied with the characteristics of each dataset.

For each dataset we constructed two analytic axes: (1) *text representation*, comparing statistical baselines to sentence encoders, and (2) *CNN architecture*, comparing shallow CNNs to a pre-trained complex CNNs. A commercial vision-language LLM (Gemini 2.0 Flash Lite) is also evaluated to show how well generative model performs on the same task. Performance is quantified with a composite retrieval score combining Mean Reciprocal Rank (MRR), median rank, Top-1, Top-5, and cosine similarity, combined with weights.

Key finding: Across all datasets, the match between the sentence embedding method and the training loss function was the most decisive factor for retrieval performance, outweighing model size.

1 Introduction

1.1 Task Overview and Motivation

This project addresses image-to-sentence retrieval: given a query image $\mathbf{x}$, the system returns the correct sentence from a fixed corpus $\mathcal{C}$. To do this, a CNN encoder is trained to map each image to a vector that is close to the corresponding sentence embedding, as measured by cosine similarity (Eq. 1).

Three structurally different datasets are generated: Type-A (geometric shapes with colour descriptions, $|C_A|{=}10,000$), Type-B (coloured MNIST digit sequences, $|C_B|{=}10,008$), and Type-C (tic-tac-toe board states, $|C_C|{=}5,269$). Each image is paired to ground-truth sentences.

A good solution requires: (i) discriminative setence embeddings that can separate different semantic sentences clearly and cluster similar sentences together in vector space; and (ii) CNN capacity matched to training datasets preventing over-fitting and embdding collapse.

1.2 Type-A: Geometric Shapes with Colour Descriptions

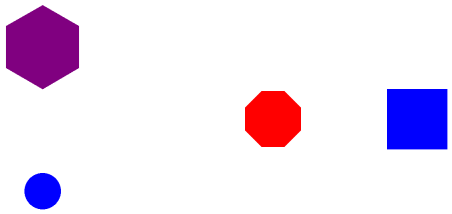


Figure 1: Sample of 2 Type-A images.

Similar in nature to the approach laid out in (Kuhnle and Copestake, 2017), type A dataset contains labelled images of shapes, varying by colour, size, shape type and the position of shapes relative to each other, which will be referred to as relation from here on. We used lists of predefined vocabulary for each variable, which were mapped to predefined sentence templates varying by stopwords and sentence order.

Sentences were generated through the random selection of size, colour, relation and object words, which were encased in a randomly selected sentence template. At this point and with the sentence decomposed into its individual parts, we then used the PIL graphics library to generate **10,000** png images.

Furthermore, our logic for the image generation was based around offsetting each shape by a fixed amount, relative to the specific relation between

the shapes in the sentence.

1.3 Type-B: Coloured MNIST Digit Sequences

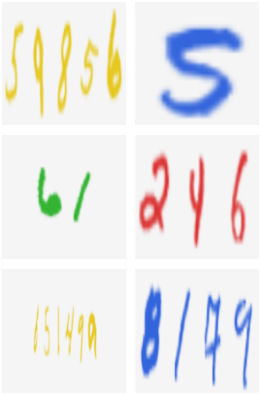


Figure 2: Sample Type-B images.

Attr.	Value	Count (%)
Size	large	5,004 (50.0%)
	small	5,004 (50.0%)
Colour	blue	2,502 (25.0%)
	green	2,502 (25.0%)
	red	2,502 (25.0%)
	yellow	2,502 (25.0%)
Digit length	1-digit	72 (0.7%)
	2-digit	720 (7.2%)
	3-digit	2,304 (23.0%)
	4-digit	2,304 (23.0%)
	5-digit	2,304 (23.0%)
	6-digit	2,304 (23.0%)

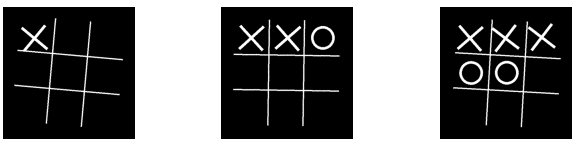
Table 1: Attribute distribution (10,008 samples).

Type-B pairs images of handwritten digit sequences with sentences. Sentences are generated with the fixed template `{size} {colour} {number}` - for example, "large blue 67".

Sentence generation pipeline Sentences are produced by enumerating all combinations of size (2), colour (4), and numbers of different digit lengths (1-6 digits with 1,261 unique numbers), without duplicates. The generated **10,008** unique sentences are distributed in the dataset as described in Table 1.

Image generation pipeline The MNIST training set (60,000 images) is first grouped by digit class (0-9). For each digit position in the target number sequence. Each 28×28 greyscale patch is tinted with the target colour (red, blue, green, or yellow) by mixing pixel intensity with the target RGB vector. The background is set to a light-grey $(245, 245, 245)$. Then the tinted patches are resized to 32×32px and stitched horizontally into a single digit sequence. Finally, all images are scaled to fill 100% (large) or 55% (small) of the 128×128 canvas and centred in the middle. The target size information `large`/`small` is therefore encoded by the relative spatial proportion of the digit sequence.

1.4 Type-C: Tic-Tac-Toe Board States



(a) $n_moves = 1$ (b) $n_moves = 3$ (c) $n_moves = 5$

Figure 3: Representative Type-C board configurations with increasing structural complexity.

Type-C concentrates on the pairs of tic-tac-toe board states and structured text. Each image is composed of a 3×3 grid, and each cell in the grid can be either X, O, or blank. The sentence that presents the spatial Layouts of Symbols through Position Specific Expressions, such as Top-Left, Middle-Right; Bottom-Centre, etc.

Each board's state internally uses a compressed form that consists solely of character codes; for instance, it could be represented in hexadecimal format as `X:TL,MR O:C,BR`. These symbolised States are transformed into grid-Based Images using X, O Markers at the corresponding positions of them.

To prevent the existence of unreasonable boards, only reachability Tic-Tac-Toe states can be retained. Enforced via move count restrictions and simple valid state checks.

2 Methods

2.0.1 Text Representations

Table 2 lists the candidate text representations used in this project. Each dataset team selected a subset of these methods for their experimental axis.

Method	Dim	Notes
TF-IDF / LSA	100	TF-IDF with Truncated SVD (Arora et al., 2017)
GloVe	300	wiki-gigaword-300, mean-pooled
Word2Vec (pretrained)	300	Google News (Mikolov et al., 2013), mean-pooled
Word2Vec (in-domain)	100	Skip-gram trained on dataset corpus (Mikolov et al., 2013)
BERT mean-pool	768	bert-base-uncased‡
BERT `[CLS]`-pool	768	bert-base-uncased‡
TinyBERT mean-pool	312	`TinyBERT_General_4L_312D`†
TinyBERT `[CLS]`-pool	312	Same model, `[CLS]` token pooling
SBERT frozen	384	`all-MiniLM-L6-v2`‡
SBERT fine-tuned	384	Same model, further trained on in-domain sentence pairs

Table 2: Candidate text representations used in this study. Each dataset team evaluated a dataset-specific subset.

2.0.2 Image Preprocessing

When training pre-trained models, all datasets reduced images down to 224×224 as to align with the input dimensions for both Resnet and GoogleNet (Szegedy et al., 2015). For type B and C however images were again reduced to 128×128 when training from scratch models. Additionally, when training pre-trained models, normalisation values from

† https://huggingface.co/huawei-noah/TinyBERT_General_4L_312D

‡ https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2; https://huggingface.co/google-bert/bert-base-uncased

the ImageNet dataset were applied and from scratch models we used mean = (0.5, 0.5,0.5), sd=(0.5, 0.5, 0.5) (Pytorch). All datasets were split using a ratio of 80:10:10.

2.0.3 Visual Encoders

The CNN architectures used in this study are summarised in Table 3. Each dataset team selected a subset of these architectures as experimental axes.Our scratch CNN model has competitive baselines since Shahinfar et al. (2020) showed that ResNet18 requires large sample counts to achieve competitive performance, and Brigato and Iocchi (2020) demonstrated that simpler networks are less likely to overfit under small data sizes.

2.0.4 Optimiser and Regularisation

All models are trained with the Adam optimiser (Kingma and Ba, 2014) ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$) with an initial learning rate of 1×10^{-5}, 1×10^{-3}, 1×10^{-4} , ReduceLROnPlateau scheduling and batch size = 32, 64, 32 respectively for each dataset. Dropout ($p = 0.3$ or $p = 0.5$) is applied at the head of each CNN model.

2.0.5 Retrieval & Training Objectives

During inference, cosine similarity is calculated with the predicted embedding against all corpus embeddings:

$$\hat{s}(\mathbf{x}) = \arg\max_j \frac{\text{CNN}(\mathbf{x}) \cdot \mathbf{e}(s_j)}{\|\text{CNN}(\mathbf{x})\|, \|\mathbf{e}(s_j)\|} \quad (1)$$

where $\mathbf{x}$ is the input image. $\text{CNN}(\mathbf{x}) \in \mathbb{R}^d$ is the predicted embedding output from the system. s_j is the j-th sentence in the corpus. $\mathbf{e}(s_j) \in \mathbb{R}^d$ is its pre-computed j-th sentence embedding, and $\hat{s}(\mathbf{x})$ is the retrieved sentence with maximum cosine similarity to the CNN output.

Two loss functions were tested in every dataset, where $\hat{\mathbf{y}}_i = \text{CNN}(\mathbf{x}_i)$ is the predicted embedding and $\mathbf{e}_i$ is the target sentence embedding:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} \left\| \hat{\mathbf{y}}_i - \mathbf{e}_i \right\|^2 \quad (2)$$

$$\mathcal{L}_{\cos} = 1 - \frac{\hat{\mathbf{y}} \cdot \mathbf{e}}{\|\hat{\mathbf{y}}\|, \|\mathbf{e}\|} \quad (3)$$

MSELoss (Mean Squared Error) minimises Euclidean distance in embedding space. **CosineLoss** (Cosine Similarity Loss) minimises angular deviation, which can directly contribute to optimising the cosine-similarity of the retrieval objective.

2.0.6 Commercial LLM

The commercial LLM model is chosen as Gemini 2.0 Flash Lite. REST API Requests are sent directly to the model endpoints using **OpenRouter** API (`https://openrouter.ai`). The prompts are designed respectively for each dataset that produce a single descriptive sentence per image. Responses were embedded with the same text representation used for each dataset's best experiment combination.

2.1 Type-A

2.1.1 Type-A Baseline

Baseline. For the CNN, we have designed a simple two layer convolutional neural network, denoted CNN-2-Layer. It consists of two convolutional blocks outputting 32 and 64 feature maps respectively. For the embedding baseline, we propose the use of TinyBert pooler embedding which takes the embedding for the [CLS], and as identified by Kumar et al, has been deemed a suitable proxy for representing the input sentence (Kumar et al., 2024). Finally the baseline loss function for type A use **CosineLoss**.

2.1.2 Type-A Axes of Experimentation

Axis 1 - CNN Architecture. This piece has identified several potentially advantageous CNN architectures from the baseline CNN-2-Layer. We implemented both CNNPool and CNNStride, explicitly trying to add more convolutional layers to capture more nuanced information. Moreover, instead of using MaxPool in CNNStride, we used stride=2 to downsample our images, enabling the CNN to learn downsampling directly, potentially more effectively encoding our images. Additionally, we also compare how greater model depth and the use of pretrained weights for both GoogLeNet and Resnet18 will affect performance.

Axis 2 - Text Representation. Representative embeddings will capture of spatial dynamics defined in the relation part of our sentence. Moreover, bag of word representations such as Word2Vec may perform poorly here given that they don't account for word order making the relation aspect difficult to identify. Moreover the mean embedding approach used in TinyBERT/BERT mean embeddings may also fail to represent this adequately as tokens are averaged irrespective of position. We have used TinyBert pooler embeddings as the baseline method, relying on the [CLS] token to capture this.

Name	Blocks	Architecture	Params	Init
CNN-1-Layer	1 conv	Conv(3→64, 3×3) → BN → ReLU → MaxPool(2×2) → AvgPool(8×8) → FC(4,096→512) → ReLU → Drop(0.3) → FC(512→d)	2.3M	scratch
CNN-2-Layer	2 conv	[Conv(ch, 3×3) → BN → ReLU → MaxPool(2×2)]$_{\times 2}$, ch = 32/64 → AvgPool(4×4) → FC(1024→512) → ReLU → Drop(0.5) → FC(512→ d)	0.74M	scratch
CNN-3-Layer	3 conv	[Conv(ch, 3×3) → BN → ReLU → MaxPool]$_{\times 3}$, ch=32/64/128 → AvgPool(2×2) → FC(512→256) → ReLU → Drop(0.3) → FC(256→d)	0.55M	scratch
CNNStride	4 conv	[Conv(ch, 3×3, stride = 2) → BN → ReLU]$_{\times 4}$, ch = 16/32/64/128 → AvgPool(3×3) → FC(1152→2500) → ReLU → Drop(0.5) → FC(2500→ d)	3.94M	scratch
CNNPool	4 conv	[Conv(ch, 3×3) → BN → ReLU → MaxPool(2×2)]$_{\times 4}$, ch = 16/32/64/128 → AvgPool(3×3) → FC(1152→2500) → ReLU → Drop(0.5) → FC(2500→ d)	3.94M	scratch
ResNet18	8 res	ImageNet backbone (He et al., 2016) (frozen) → FC(512→256) → ReLU → FC(256→d)	11.4M	pretrained
GoogLeNet_V1	9 modules	GoogLeNet backbone (Szegedy et al., 2015) (frozen) → FC(1024→512) → ReLU → FC(512→d)	6.8M	pretrained

Table 3: Visual encoder architectures. d = target sentence embedding dimension.

2.1.3 Type-A Proposed Improvements

Improvement 1: Model Architecture. This piece has identified several issues with the initial CNN-2-Layer model. We therefore propose the use of Resnet18 to provide greater depth, and pretrained weights which we will fine tune. Moreover we opted against the selection of the GoogleNet model as given the relative simplicity of our images, it should not require significant depth to extract a representative image encoding, and thus the additional computation would not be required.

Improvement 2: Embeddings. We propose the use of SBERT to directly capture spatial information in the sentence, and thus better enable the linear layers in the CNN to effectively map images to embeddings. The ability to capture this spatial information should enable the embeddings to more clearly represent how each shape is positioned relative to the other thus evidencing a more recognisable pattern for the model to learn.

2.2 Type-B

2.2.1 Type-B Baseline

The CNN-1-Layer and TF-IDF/LSA combination is chosen as a baseline (B0), which is the simplest retrieval system that requires no pretrained external model.

A TF-IDF matrix is computed for the 10,008 sentences in the entire dataset. While all sentences follow a fixed three-token format, the vocabulary consists of 1,267 unique tokens. As a result, the term frequency of every present token is effectively uniform (TF $\approx \frac{1}{3}$), making IDF as the main component of discrimination. While size and colour tokens are frequently used, each number identity appears in at most 8 documents, IDF(number) = 4.05. The SVD exploration showed first five components capture 8.7% cumulative variance, and from $k = 6$, the marginal gain is constant to 7.36×10^{-4}. This implies that most of the embeddings in Type-B are orthogonal to each other. Thus, increasing the size of the target embedding only causes computational overhead during training. $k = 100$ is therefore selected which can capture 15.7% of total variance and yields 80.1 training samples per output dimensions.

2.2.2 Type-B Axes of Experimentation

Axis 1 - Text representation. The sentence template `{size} {colour} {number}` has a small, structured vocabulary. Pretrained models (GloVe, Word2Vec) may fail to distinguish number tokens because such tokens are rare or absent in general-domain dataset. We hypothesise that contextual encoders (TinyBERT, SBERT) will produce better separated embeddings, improving retrieval rank.

Axis 2 - CNN architecture. Type-B images encode three independent attributes: colour, size, and digit identity. Shallow neural networks with large spatial receptive fields (CNN-1-Layer) may capture colour and size more effectively, while deeper networks (CNN-3-Layer) may better resolve digit patterns. ResNet18 is expected to be over-parameterised compared to the 8,006 training set size (Shahinfar et al., 2020).

2.2.3 Type-B Proposed Improvements

Improvement 1: Contextual sentence encoders (SBERT, TinyBERT). We choose SBERT and TinyBert, expecting that contextual encoding can discriminatively separate for number emebddings in the target space compared to TF-IDF/LSA. This will allow a reduction in rank errors between sentence embeddings that share colour and size but differ only in digit value.

Improvement 2: L2 normalisation for target

emebeddings. Since retrieval process use cosine similarity (Eq. 1), all target embeddings are normalised before cosine similarity is calculated. Thus, training with raw unnormalised target embeddings can introudce an inconsistency. We address this by L2-normalising all the sentence embeddings before the training process, allowing the MSELoss objective to work on the same geometry as the retrieval metric.

Improvement 3: SBERT fine-tuned (384-dim). `all-MiniLM-L6-v2` is further fine-tuned on Type-B sentence pairs via contrastive (MultipleNegativesRanking) training. 50,000 random sentence pairs are sampled from Type-B vocabulary, each labelled with its Jaccard word-overlap score, and the model is trained for 3 epochs using CosineLoss.

2.3 Type-C

The Type-C baseline is designed as a simple and fully controlled image-to-sentence retrieval pipeline without pretrained components. We adopt a shallow CNN-1-Layer as the visual encoder, combined with TF-IDF sentence embeddings and Mean Squared Error (MSE) loss.

CNN-1-Layer provides a low-capacity model that captures basic spatial patterns from 3×3 board images, serving as a lower-bound reference for evaluating the impact of increased model capacity. TF-IDF is selected as a deterministic baseline that preserves token-level distinctions between positional terms (e.g., "top left" vs. "middle right") without introducing semantic smoothing.

The model is trained to regress image embeddings towards sentence embeddings using MSE loss, while retrieval is performed using cosine similarity (Eq. 1). This setup provides a minimal and interpretable baseline for analysing the effects of representation and model design.

2.3.1 Type-C Axes of Experimentation

Axis 1 - Text representation. Type-C requires distinguishing between sentences that differ only in Fine-grained Positional Tokens. We compare TF-IDF, GloVe and SBERT. TF-IDF will perform better because it has higher sensitivity at the token level; GloVe provides a reference state of discrimination and the structure of semantics. SBERT is expected to perform poorly; that is, it may underperform. Space will narrow the distance between different Places.

Axis 2 - CNN architecture. Type-C requires distinguishing between sentences that differ only in Fine-grained Positional Tokens. We compare TF-IDF, GloVe and SBERT. TF-IDF will perform better because it has higher sensitivity at the token level; GloVe provides a reference state of discrimination and the structure of semantics. SBERT is expected to perform poorly; that is, it may underperform. Space will narrow the distance between different Places.

Establishing a benchmark followed by proposing these three targeted improvements.

Improvement 1: Text representation. We further add GloVe and SBERT in TF-IDF to compare with them. Token-level and semantically smoother Embeddings. To see if Semantic Richness Benefits or Hinders Structured Spatial Matching.

Improvement 2: Model capacity. Replace CNN-1-layer with CNN-3, and ResNet-18 for enhancing spatial feature extraction. And evaluate the impact of Model Capacity on the Alignment Performance.

Improvement 3: Complexity-aware evaluation. The performance changes with an increasing number of steps, i.e., (n_moves). Insight on model behaviour as the number of nodes grows large.

2.4 Performance Metrics

Performance is quantified by a composite score that aggregates five retrieval metrics.

$$\begin{aligned}\text{composite} = {} & W_1 \cdot \text{MRR}_{\text{norm}} + W_2 \cdot \text{MedRank}_{\text{norm}} \\ & + W_3 \cdot \text{Top-1}_{\text{norm}} + W_4 \cdot \text{Top-5}_{\text{norm}} \\ & + W_5 \cdot \text{MeanCosine}_{\text{norm}}\end{aligned} \tag{4}$$

where $\sum_{i=1}^{5} W_i = 1$. Each metric evaluates a distinct feature of retrieval performance: **MRR** (Mean Reciprocal Rank) rewards models that rank the correct answer highest. This averages the reciprocal of the answer's rank. **Median Rank** reports the median position of the correct item. **Top-1** and **Top-5** are the proportions of queries for which the correct sentence at rank 1 or within the top 5, and **Mean Cosine Similarity** measures geometric alignment between the predicted and target embeddings. Under the retrieval task and the use of mixed text representations, which yield embeddings carrying both directional and magnitude components, the weights **(0.35, 0.25, 0.20, 0.15, 0.05)** are assigned to prioritise rank-based metrics while still considering for embedding-space alignment. MRR receives the highest weight (W_1=0.35) as it detects rank quality across all test samples. The mean cosine

component is down-weighted (W_5=0.05) because artificially high cosine values can be the signals of collapsed models.

Median rank is normalised as:

$$\text{MedRank}_{\text{norm}} = 1 - \frac{\text{median_rank}}{|C|} \quad (5)$$

so that rank 1 maps to ≈1.0 and rank $|C|$ maps to 0.0, where $|C|$ is the full corpus size at retrieval time.

2.5 Experimental Setup

2.5.1 Type-A Setup

The Type-A dataset contained 10,000 image-sentence pairs and was split into train, validation, and test sets using an 80:10:10 ratio, giving 8,000 training samples, 1,000 validation samples, and 1,000 test samples. The same split was reused across all runs for reproducibility. Validation loss was used for checkpoint selection and early stopping, and final metrics were computed on the held-out test set. During evaluation, predicted image embeddings were ranked against the relevant Type-A sentence corpus for each experiment.

2.5.2 Type-B Setup

For Type-B, any run where the mean cosine value is over 0.90 with colour-size correctness falling below 0.95 receives a 0.0 for composite score (hard collapse penalty).

The 0.95 threshold was fixed after Stage 1 runs: non-collapsed runs achieve correctness ≥ 0.998 (e.g. S1an, S1e, S2b), while every collapsed run scores ≤ 0.687 (e.g. Word2Vec in-domain, TinyBERT [CLS]-pool). Thus, within the interval of $[0.687, 0.998]$, we chose a conservative threshold 0.95 close to the non-collapsed level.

The penalty is needed because mean cosine alone is an unreliable collapse detector: Word2Vec (in-domain) achieved cosine > 0.999 but correctness only 0.26–0.53, indicating dimensional collapse rather than genuine retrieval.

The 10,008 Type-B samples were split into train (80%=8,006), validation (10%=1,001), and test (10%=1,001) using stratified sampling on digit length (`n_digits`), seed 42. Stratification method allows the rare 1-digit (0.7%) and 2-digit (7.2%) classes to be proportionally represented in every split.

All experiments ran on Google Colab (NVIDIA T4 GPU). The sentence embeddings were precomputed once per method and stored as `.pt` files so that the CNN training loop loads fixed targets and guarantees retrieval-time targets are identical.

2.5.3 Type-C Setup

For type-c, all experiments were carried out on the 5,269 reach-able tic-tac-toe images - texts pairs using a shared train-validation-test split of 80:10:10.

The type-C experimental grid comprised of three kinds of information encoders and presented in the following manner: CNN-1, CNN-3, Resnet-18 combined with TF-IDF(16-dimension), GloVe(100-dimension), and SBERT(384-dimensional). All models were trained using fixed pre-computed sentence embeddings as the regression target, validation loss was used to select checkpoints, and the final retrieval performance on the held-out test set was evaluated. The results are divided into groups according to the numbers of occupied boards and moves; Test whether Retrieval changes as Structural Complexity Increases.

2.6 Results

2.6.1 Type-A Results

The Table 4 shows how Type-A experiments compared multiple CNN architectures, sentence embedding types, and loss functions using retrieval-based metrics. Runs were ranked using a composite score based on Top-1, Top-5, MRR, median rank, and mean cosine similarity.

The strongest Type-A configuration was **CNNPool + SBERT + CosineLoss**, followed by **ResnetDSA + SBERT + CosineLoss**. Among the non-SBERT runs, **ResnetDSA + Word2Vec-pretrained + CosineLoss** performed best. Overall, SBERT was the most effective sentence embedding for Type-A, especially when paired with CosineLoss, while CNNPool gave the best result among the evaluated image encoders.

CosineLoss outperformed MSELoss for most embedding families. However, some pooler-based runs still achieved high cosine similarity with weak retrieval performance, showing that embedding closeness alone does not guarantee correct semantic retrieval.

2.6.2 Type-B Results

Table 5 shows the combined results of Stage 1 (embedding methods with CNN-1-Layer fixed) and Stage 2 (CNN architectures using the top embeddings from Stage 1), ranked by composite score across all 33 runs. Collapsed runs (composite 0.0) are grouped at the bottom of each stage.

Stage	Overall Rank	Run	Model	Embedding (dim)	T@1	T@5	MRR	Med.Rk	MeanCos.	Comp.
Stage 1	1	1_CNNPool_sbert_CosineLoss	CNNPool	SBERT (384)	0.043	0.237	0.151	15.0	0.861	**0.989**
	2	4_ResnetDSA_sbert_CosineLoss	ResnetDSA	SBERT (384)	0.042	0.170	0.118	29.0	0.836	0.862
	4	2_CNNStride_sbert_CosineLoss	CNNStride	SBERT (384)	0.016	0.071	0.062	44.0	0.841	0.545
	5	0_CNN2Layer_sbert_CosineLoss	CNN2Layer	SBERT (384)	0.008	0.029	0.032	161.0	0.802	0.379
	6	3_GoogleNet_sbert_CosineLoss	GoogleNet	SBERT (384)	0.002	0.026	0.023	210.5	0.728	0.313
Stage 2	3	9_ResnetDSA_P_wvec_CosineLoss	ResnetDSA	Word2Vec-pre (300)	0.034	0.120	0.087	77.5	0.888	0.712
	7	6_ResnetDSA_TB_mean_CosineLoss	ResnetDSA	TinyBERT-mean (312)	0.004	0.018	0.017	396.0	0.872	0.264
	8	14_ResnetDSA_B_mean_MSELoss	ResnetDSA	BERT-mean (768)	0.002	0.009	0.013	341.0	0.879	0.254
	9	8_ResnetDSA_B_mean_CosineLoss	ResnetDSA	BERT-mean (768)	0.002	0.010	0.011	418.5	0.873	0.231
	10	5_ResnetDSA_TB_pooler_CosineLoss	ResnetDSA	TinyBERT-pooler (312)	0.002	0.008	0.010	456.5	0.972	0.223
	11	13_ResnetDSA_B_pooler_MSELoss	ResnetDSA	BERT-pooler (768)	0.002	0.007	0.009	472.0	0.968	0.215
	12	15_ResnetDSA_P_wvec_MSELoss	ResnetDSA	Word2Vec-pre (300)	0.000	0.009	0.010	428.0	0.725	0.207
	13	7_ResnetDSA_B_pooler_CosineLoss	ResnetDSA	BERT-pooler (768)	0.002	0.006	0.009	499.0	0.970	0.207
	14	12_ResnetDSA_TB_mean_MSELoss	ResnetDSA	TinyBERT-mean (312)	0.001	0.009	0.009	463.5	0.830	0.207
	15	10_ResnetDSA_sbert_MSELoss	ResnetDSA	SBERT (384)	0.002	0.005	0.009	475.5	0.487	0.188
	16	11_ResnetDSA_TB_pooler_MSELoss	ResnetDSA	TinyBERT-pooler (312)	0.001	0.006	0.007	513.0	0.558	0.175

Table 4: Combined Stage 1-2 overall ranks by composite score across all 16 Type-A runs. T@1/T@5 = Top-1/Top-5 accuracy; MRR = Mean Reciprocal Rank; Med.Rk = median retrieval rank; MeanCos. = mean cosine similarity; Comp. = composite score.

Stage	Overall Rank	Run	Model	Embedding (dim)	T@1	T@5	MRR	Med.Rk	MeanCos.	Comp.
Stage 1	1	S1an	CNN-1	SBERT†-normed (384)	0.012	0.029	0.031	249	0.917	**0.964**
	3	S1e	CNN-1	TinyBERT-mean (312)	0.006	0.029	0.028	169	0.966	0.830
	5	S1bn	CNN-1	SBERT-ft†-normed (384)	0.006	0.020	0.019	358	0.871	0.686
	6	S1b	CNN-1	SBERT-ft (384)	0.003	0.021	0.018	351	0.871	0.624
	7	S1en	CNN-1	TinyBERT-mean†-normed (312)	0.004	0.014	0.017	262	0.962	0.612
	9	S1a	CNN-1	SBERT (384)	0.002	0.006	0.009	510	0.375	0.419
	8	B0	CNN-1	TF-IDF/LSA (100)	0.004	0.006	0.009	639	0.796	0.463
	10	B0n	CNN-1	TF-IDF/LSA†-normed (100)	0.001	0.004	0.006	642	0.798	0.381
	-	TinyBert [CLS]-pool, TF-IDFxWord2Vec (in-domain), Word2Vec (in-domain) (various; colour_size_corr < 0.95)								0.0‡
Stage 2	2	S2dn	CNN-3	SBERT†-normed (384)	0.009	0.035	0.031	261	0.916	**0.935**
	4	S2b	CNN-3	TinyBERT-mean (312)	0.004	0.031	0.026	173	0.966	0.791
	-	Both ResNet18 models trained with SBERT-normed and TinyBert-mean (various; colour_size_corr < 0.95)								0.0‡

Table 5: Combined Stage 1-2 overall ranks by composite score across all 33 Type-B runs. Only the main runs are kept for clarity. † L2-normalised targets. ‡Hard-collapsed runs (colour_size_correct < 0.95); composite score forced to 0.0.

The most notable result is the gap between S1an (SBERT†, composite 0.964) and S1a (non-normalised SBERT, composite 0.419). Although the architecture and encoder are identical, L2-normalisation combined with CosineLoss more than doubled the composite score. This shows that aligning the training objective with the cosine-similarity retrieval metric substantially improves performance.

Figure 4 helps explain why TinyBERT-mean achieves strong retrieval without normalisation: its t-SNE projection forms clearly separated clusters by digit length, whereas SBERT embeddings show overlapping blobs. Fine-tuned SBERT (S1bn/S1b) underperformed frozen SBERT, suggesting that 8,006 sentence pairs were insufficient (Brigato and Iocchi, 2020).

CNN-3-layer with SBERT† (S2dn, composite 0.935) slightly underperformed CNN-1-layer with SBERT† (S1an, 0.964). This is likely because CNN-1-Layer's AvgPool(8×8) retains a 4,096-dimensional spatial representation, whereas CNN-3-Layer's AvgPool(2×2) reduces it to 512 dimensions. For Type-B, where the key features are mostly global (colour and size), spatial retention appears to matter more than depth.

All four ResNet18 variants collapsed to composite 0.0, suggesting that an over-parameterised pretrained backbone is not well-suited, and ImageNet visual priors are poorly matched to synthetic digit images (Shahinfar et al., 2020).

2.6.3 Type-C Results

Across all model architectures, GloVe-based representations achieved the strongest Top-1, Top-5, and MRR values. The TF-IDF method demonstrated an average result; however, the SBERT-based configuration yielded a lower score and thus was least effective in representing positions of Type-C compared with other methods.

Performance also varied across image encoders. CNN-3 had a significantly higher Top-1 Accuracy and lower Mean Rank than CNN-1 across all evaluation results; ResNet18 also failed to outperform

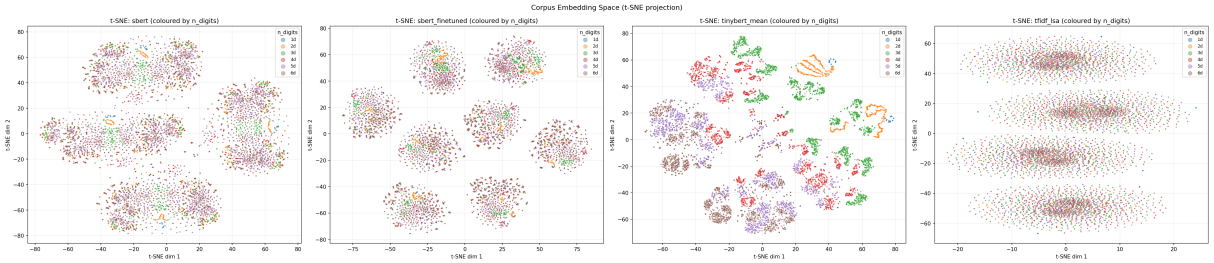


Figure 4: t-SNE projections of the Type-B corpus embedding space, coloured by digit length (`n_digits`). TinyBERT-mean (third panel) shows well-separated clusters per digit length. SBERT, SBERT-finetuned (first two panels), and TF-IDF/LSA (rightmost panel) produce overlapping blobs across digit clusters.

Stage	Rank	Run	Model	Embedding (dim)	T@1	T@5	MRR	Med.Rk	MeanCos.	Comp.
Stage 1	5	C1-SBERT	CNN-1	SBERT (384)	0.076	0.211	0.159	23	0.996	0.480
	6	C1-TF-IDF	CNN-1	TF-IDF (16)	0.073	0.173	0.135	41	0.973	0.433
	7	C1-GloVe	CNN-1	GloVe (100)	0.047	0.167	0.124	32	0.999	0.417
Stage 2	1	C3-GloVe	CNN-3	GloVe (100)	0.295	0.748	0.488	3	0.999	**1.000**
	2	R-GloVe	ResNet18	GloVe (100)	0.233	0.588	0.397	4	0.999	0.857
	3	C3-TF-IDF	CNN-3	TF-IDF (16)	0.206	0.461	0.336	7	0.984	0.764
	4	R-TF-IDF	ResNet18	TF-IDF (16)	0.139	0.338	0.238	16	0.976	0.612
	8	C3-SBERT	CNN-3	SBERT (384)	0.039	0.142	0.095	79	0.998	0.342
	9	R-SBERT	ResNet18	SBERT (384)	0.033	0.102	0.082	74	0.993	0.324
S3	10	CLIP-B32	CLIP ViT-B/32	Joint Image-Text	0.002	0.011	0.013	272	0.299	–

Table 6: Type-C retrieval performance across model and embedding configurations, including a pretrained CLIP zero-shot baseline.

simple CNN-based architectures consistently. Table 6 shows the MRR values for different configurations. GloVe-based models obtain higher MRR compared to TF-IDF and SBERT, indicating that relevant matches are ranked closer to the top of the retrieval list.

2.7 Commercial LLM Comparison

2.7.1 Type-A: LLM Comparison

The model was prompted to produce one sentence per image using the same constrained vocabulary and sentence-template style as the dataset. The generated outputs were then embedded using SBERT and retrieved against the full Type-A sentence corpus, so that evaluation matched the retrieval protocol used in the main experiments.

Run	T@1	T@5	MRR	MeanCos.	MeanRk	Med.Rk
Gemini + SBERT retrieval	0.179	0.527	0.340	0.977	12.911	5.0
Best Retrieval System (CNNPool+SBERT+Cosine)	0.043	0.237	0.151	0.861	15.0	15.0

Table 7: Type-A: commercial LLM vs. best retrieval system on the test set.

On the 1,000-image Type-A test set, Gemini achieved 0.179 Top-1, 0.527 Top-5, and 0.340 MRR, outperforming the best supervised CNN baseline on retrieval. However, exact sentence match was 0.044 and semantic match rate was 0.366. This suggests that the LLM often produced a close description, but not the exact structured sentence required for perfect retrieval.

2.7.2 Type-B: LLM Comparison

The LLM output was embedded with the best-performing text encoder (SBERT-normed, TinyBERT mean-pool) before cosine-similarity retrieval. Two prompt designs were compared: a *detailed* prompt providing clear formatting instructions and a *minimal* prompt providing only the bare task guidance. Results on whole test set (1,001) are reported in Table 8.

Run	Prompt	T@1	MRR	col_size_corr
p1-LLM-sbert-norm	Detailed	0.454	0.503	0.523
p1-LLM-tinybert	Detailed	**0.458**	0.461	0.523
p2-LLM-sbert-norm	Minimal	0.144	0.166	0.530
p2-LLM-tinybert	Minimal	0.144	0.146	0.515
Best Retrieval System (S1an)	-	0.012	0.031	1.0

Table 8: Type-B: commercial LLM vs. best retrieval system on the test set. col_size_corr=fraction of outputs with correct colour and size tokens.

The detailed-prompt LLM achieved Top-1 ≈45.5%, nearly 38 times higher than the best CNN (1.2%). Prompt design was crucial in this task since switching to a minimal prompt Top-1 rate decreased to 14.4%. This suggests that the main limitation was not raw visual recognition, but the model's dependence on explicit formatting instructions for reliable template compliance.

2.7.3 Type-C: LLM Comparison

The model was prompted to output descriptions of tic-tac-toe board images, which were then embedded using SBERT and matched against the Type-C sentence corpus via cosine-similarity retrieval. For comparison, the best-performing model (CNN3-GloVe) is included.

Run	Prompt	Embedding	test size	T@1	T@5	MRR
LLM-openrouter	Detailed	SBERT	660	0.006	0.018	0.014
Best Retrieval System (CNN3-GloVe)	–	GloVe	528	0.295	0.748	0.488

Table 9: Comparison between the commercial LLM baseline and the best supervised retrieval model on Type-C.

The commercial LLM achieved only 0.006 Top-1 accuracy, 0.018 Top-5 accuracy, and an MRR of 0.014. Although its mean cosine similarity was high (0.988), retrieval performance remained weak, indicating that the generated descriptions were often semantically similar but not structurally identical to the canonical board-state sentences.

These findings suggest LLMs remain limited on symbolic spatial reasoning tasks requiring precise structured outputs, whereas domain-specific supervised retrieval models are considerably more reliable for exact board-state matching.

2.8 Error Analysis

2.8.1 Type-A Error Analysis

A recurring error pattern in Type-A was the retrieval of sentences that preserved some surface attributes correctly, such as colour or shape type, while failing to recover the correct spatial relation between the two objects. In these cases, the retrieved sentence often contained the same vocabulary items as the target sentence, but assigned them to the wrong relation or reversed the object pairing.

This problem was especially visible in runs using pooler-based embeddings. Although several of these runs achieved high mean cosine similarity, they still performed poorly on Top-1, Top-5, and full-structure match. This suggests that the corresponding embedding spaces were not sufficiently discriminative for sentence retrieval, even when the predicted vectors appeared numerically close to the targets. In effect, these models often captured partial attribute similarity without preserving full sentence structure.

A second source of error came from confusion between sentences that differed only in the assignment of attributes to the two shapes. Since Type-A uses highly regular templates and a small controlled vocabulary, many sentences are lexically similar. This means that a model can retrieve a sentence with mostly correct words while still producing a semantically incorrect description. These errors reinforce the importance of using evaluation metrics that go beyond cosine similarity and explicitly measure relation correctness and structural recovery.

2.8.2 Type-B Error Analysis

Runs using TinyBERT `[CLS]` pooler, pretrained Word2Vec, skip-gram Word2Vec, and TF-IDF-weighted Word2Vec all collapsed. The best checkpoints were reached within 6-11 epochs, mean cosine > 0.993, median rank 629 to 1,730. Under $\mathcal{L}_{\text{MSE}}$ (Eq. 2), gradients from uninformative dimensions can push the network toward the mean embedding, without sufficiently penalising retrieval errors.

From LLM test, of 498 truly large images, 477 (95.8%) were described as "small". Colour prediction was 100% accurate. A wrong size token redirects retrieval to a different sentence entirely, resulting in a col_size_corr of 0.523 and an effective Top-1 of about 46%. A likely cause is that 128×128 images appear uniformly small in the LLM's visual context. The minimal LLM prompt produced 640/1,001 outputs (63.9%) with non-numeric digit strings. This happens to $\geq$4-digit numbers (4-digit: 4.8%, 5-digit: 5.6%, 6-digit: 7.0%, 1-3-digit: 0%), confirming that long sequences in a digit exceed the LLM's default formatting ability.

2.8.3 Type-C Error Analysis



Figure 5: Mean cosine similarity and Top-1 accuracy vs. number of moves on Type-C.

All models' cosine similarity is above 0.98 across various settings of the retrieval function failure. As shown in Figure 5, similarity values increase slightly with the number of moves, while Top-1 accuracy remains low across all ranges. This indicates that cosine similarity is not sensitive to small positional differences. For example, descriptions that differ by a single location may still produce similar embeddings, despite representing different board states.

Models using SBERT embeddings achieve cosine similarity scores comparable to other methods, but show consistently lower retrieval performance. This suggests that sentences describing

different board states may be mapped to nearby regions in the embedding space. In contrast, lower-dimensional embeddings such as GloVe exhibit a wider distribution, which allows for greater separation between different configurations.

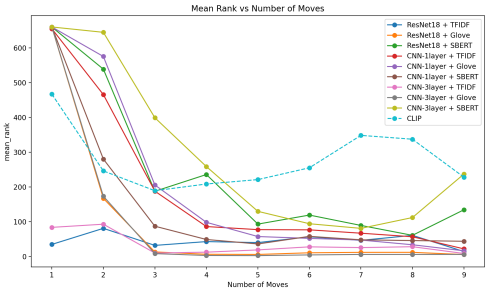


Figure 6: Mean rank vs. number of moves on Type-C.

The relationship between board complexity and retrieval performance is shown in Figure 6. As the number of moves increases, mean rank does not show a consistent decreasing trend. This may be due to the increasing number of possible configurations as more positions are filled, leading to a higher likelihood of similar but incorrect matches.

3 Conclusions

Type-A key findings. The results show that sentence-level embeddings which preserve full contextual and relational information are more effective than simpler pooler-style representations for this task. A key finding in Type-A is that many sentences contain similar vocabulary but differ critically in spatial structure, meaning that high embedding similarity alone is not enough. Some runs achieved strong cosine scores while still performing poorly on retrieval and structural correctness. This shows that effective Type-A evaluation must go beyond numerical similarity and include semantic comparison metrics that reflect the true meaning of the sentence.

Type-B key findings. The key finding is the impact of L2-normalising target embeddings. For SBERT, normalisation doubled the composite score with no structural change, only a cosine-aligned training. We were able to conclude that we should *align the training objective with the retrieval metric*. However, the t-SNE projection (Figure 4) shows that TinyBERT-mean forms well-separated clusters by digit length, even though it is not cosine-aligned, demonstrating that structured embedding space still can support competitive retrieval performance.

Type-C key findings. The results demonstrate that retrieval performance on structured spatial tasks depends primarily on the compatibility between the representation space and the underlying combinatorial structure. Token-level embeddings, such as GloVe outperform semantically smooth representations because they preserve discrete positional distinctions required to differentiate board states.

What worked. Across all datasets, the best results were obtained by matching the optimal text representation that supports complexity of the target sentences. Type-A's relational descriptions and Type-B's simple fixed sentences both benefited from contextual encoders (SBERT). Type-C's discrete positional vocabulary benefited from token-level representations (GloVe) that maintains positional distinctions. CosineLoss or L2 normalisation of targets played crucical role for the whole datasets by aligning with consine-based retrival metrics.

What did not work. A shared failure across all three datasets is the contradiction between mean cosine similarity and retrieval accuracy. The predictions often showed with high cosine scores with poor retrieval accuracy. Especially for Type-B and Type-C, the embedding collapse was the dominant failure, which is driven by MSELoss on low-variance target embeddings. Contextual encoders (SBERT), which are powerful in semantic processing, performed poorly on structural discrimination (Type-C), and over-parameterised CNN backbones (ResNet18) degraded retrieval on short sentences (Type-B).

Open challenges. The commercial LLM results reveal an important gap between approximate semantic description and exact structured retrieval. In Type-A, Gemini improved retrieval performance substantially over the best supervised CNN baseline, but exact sentence recovery remained limited. In Type-B, the LLM outperformed the best supervised CNN, but with 50% size error. For type-C, the LLM still failed on tasks requiring exact structural fidelity. Together, these results suggest that current sentence-embedding CNN pipelines remain weaker than generative vision-language models on visually descriptive tasks, but generative models are still unreliable when precise symbolic structure must be preserved. Closing this gap is an open challenge that likely requires stronger multi-modal alignment, contrastive pre-training, or hybrid systems that combine structured retrieval objectives with richer visual representations.

References

Sanjeev Arora, Yingyu Liang, and Tengyu Ma. 2017. A simple but tough-to-beat baseline for sentence embeddings.

Lorenzo Brigato and Luca Iocchi. 2020. A close look at deep learning with small data. *Preprint*, arXiv:2003.12843.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Diederik P. Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *Preprint*, arXiv:1412.6980.

Alexander Kuhnle and Ann Copestake. 2017. ShapeWorld: A new test methodology for multimodal language understanding. *arXiv preprint arXiv:1704.04517*.

Ajay Kumar, Nilesh Ware, and Shaurya Gupta. 2024. Leveraging Transfer Learning: Fine-Tuning methodology for Enhanced Text Classification using BERT. In *2024 IEEE Pune Section International Conference (PuneCon)*, pages 1–5.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. *Preprint*, arXiv:1301.3781.

Pytorch. Models and pre-trained weights — Torchvision 0.26 documentation. https://docs.pytorch.org/vision/stable/models.html.

Saleh Shahinfar, Paul Meek, and Greg Falzon. 2020. "how many images do i need?" understanding how sample size per class affects deep learning model performance metrics for balanced designs in autonomous wildlife monitoring. *Ecological Informatics*, 57:101085.

Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2015. Going deeper with convolutions. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9, Boston, MA, USA. IEEE.